Einführung in **R**

Contents

Einleitung **3**

1 R Grundlagen **5**

1.1 Was ist **R**? . . . 5

1.2 Warum **R**? . . . 5

1.3 Installation . . . 6

2 Interaktion mit R und RStudio **8**

2.1 Das User Interface von RStudio . . . 8

2.2 Die Console-Oberfläche . . . 9

2.3 Die Script-Oberfläche . . . 11

Shortcuts . . . 11

2.4 Die Environment-Oberfläche . . . 12

2.5 Die Output-Oberfläche . . . 14

3 Objekte in R-Programmierung **16**

3.1 Erstellen eines Objekts . . . 16

3.2 Namen von Objekten . . . 18

3.3 Verändern von Objekten . . . 18

4 Datenstrukturen in R **20**

4.1 Vektoren . . . 21

4.2 Listen . . . 22

4.3 Matrizen und Arrays . . . 24

5 Grundlegende R-Funktionen **28**

5.1 Vektorisierte mathematische Operationen 28

5.2 Vektorisierte Matrix-Operationen 29

5.3 **R** Operatoren 30

5.4 Missing Values 33

5.5 Packages 33

6 Funktionen und Kontroll-Strukturen **35**

6.1 For-Loops 35

6.2 If-Else 36

6.3 While-Loops 37

6.4 Eigene Funktionen 37

7 Daten **40**

7.1 Einlesen von Daten 40

7.2 Das readr-Package 41

7.3 Speichern von Daten 41

7.4 Daten Selektion 42

8 Plotten **49**

8.1 Plot-Funktion 49

8.2 Symbole 51

8.3 Low-Level Plot-Funktionen 53

8.4 points() und Segments() 54

8.5 abline() und segments() 55

8.6 legend() 57

8.7 Multiple Plots 59

Einleitung

Dieses Skript ist als Teil der Bachelorveranstaltung Statistik der Wirtschaftswissenschaftlichen Fakultät der Universität Göttingen entstanden und soll Studenten helfen auf eine einfache und direkte Art und Weise die wichtigsten Funktion der Statischen Programmiersprache **R** zu verstehen und anwenden zu können. Der Inhalt soll des Weiteren allen Studierenden helfen welche Interesse oder Fragen hinsichtlich **R**-Programmierung haben.

Die aktuellste Version des Skriptes, sowie die unterliegenden Source-Files, sind im Gitlab-Repository "intro2r" zu finden.

Dieses Skript wurde in Rmarkdown und dem learnr-Package geschrieben und erstellt. Es handelt sich hierbei um einen Work-in-Progress und wird ständig durch Mitarbeiter des Lehrstuhl Statistik der Universität Göttingen erweitert und verbessert. Daher würden die Autoren sich sehr über Verbesserungsvorschläge, neue Ideen oder Fehlermeldungen freuen. Hierfür einfach eine E-Mail an folgende Adresse:

gitlab+kruse44-intro2r-10509-issue-@gwdg.de

Das folgende Lehrmaterial ist und wird immer frei sein und kann unter einer CC-BY-SA 4.0 Lizenz verwendet, verbreitet und modifiziert werden. Der Hauptautor der originalen Version ist René-Marcel Kruse (Github@RMKruse, gitlab.gwdg@kruse44) unter Mitwirkung von:

- Jasmin Schilling

Dieses Skript bezieht sich auf verschiedene Quellen, deren inhaltliche Aufarbeitung und Darstellung der Thematik, als Grundlage und Bezugspunkt beim Erstellen dieses Skriptes dienten. Hierbei sei vor allem auf folgende Quellen verwiesen, wobei es sich bei allen Quellen um Free-and-Open-Source Lehrinhalte der jeweiligen Autoren handelt:

- Advanced R von Hadley Wickham
- Merely Useful: Novice R von Madeleine Bonsma-Fisher et al.
- R for Data Science von Hadley Wickham und Garrett Grolemund

- Hands-On Programming with R von Garrett Grolemund
- Fundamentals of Data Visualization von Claus O. Wilke
- YaRrr! The Pirate's Guide to R von Nathaniel D. Phillips

Des Weiteren diente als Grundlage die Vorlesungsfolien des Statistik-Master Kurses "Introduction to Statistical Programming" von Paul Wiemann.

Chapter 1

R Grundlagen

1.1 Was ist R?

R ist eine objekt-orientierte und funktionale Programmiersprache welche für die Analyse von Daten entwickelt worden ist und heute den de-facto Standard in der Statistk und vielen quantiativen Bereichen in der Wissenschaft darstellt.

Aber was heißt das? Und vor allem was beudetet es für dich? Und wieso sollte es dich interessieren **R** zu lernen?

1.2 Warum R?

Nun ist die Frage, wieso sollte man **R** anstatt vieler andere Programmiersprache (MATLAB) oder Programmen für Datenanalyse (SPSS) oder Point-and-Click-Adventure (STATA) nutzen?

Hier für gibt es viele Gründe, einige der wichtigsten sind die folgenden:

1. **R** ist zu 100% frei und opensource. Das bedeutet für dich als Nutzer, dass du kein Geld bezahlen musst um R zu nutzen. Desweiteren gibt es eine riesen große Community online, welche das Projekt unterstützt, weiterentwickelt und sich gegenseitig bei Fragen hilft.
2. **R** hat sich vor allem in der akademischen Welt als die Programmiersprache für Statistik und Datenanalyse durchgesetzt. R erlaubt es (vor allem im Zusammenspiel mit IDEs wie Rstudio) deine Gedanken und Modelle in Code umzuwandeln welcher sofort von der Maschine ausgeführt werden kann. Desweiteren im Gegensatz zu anderen Programmiersprachen wie C++/Fortran muss **R** Code nicht immer kompelliert werden und ist dadurch viel benutzerfreundlicher.

3. Für dich als Einsteiger ist vor allem Rstudio, der meist genutze IDE (Integrated development environment) für **R**, ein Vorteil da dieser es erlaubt einfach und interaktiv Code zu schreiben sowie Analysen durchzuführen.
4. **R** als Sprache und Rstudio als IDE sind ebenfalls extrem vielseitig, da sie es dir erlauben neben klassischen Datenanalysen und Grafiken auch interaktive Webseiten oder Dokumente zu verfassen. (Dieses gesamte Dokument wurde mit Hilfe von R geschrieben)

Nun kennst du vielleicht ebenfalls andere Programmiersprachen welche für Statistik und Data Science genutzt werden, hierbei kommen vor allem Python und Julia in den Sinn. Wir wollen dir auf keinen Fall sagen, dass es nur die eine richtige Programmiersprache gibt die du lernen musst und das **R** in allen Belängen die Beste ist. In der Wirklichkeit nutzen Programmierer, Statistiker oder die Doktoranden am Lehrstuhl eine Mix aus verschiedenen Programmiersprachen um ihre Aufgaben zu lösen. Wir sind allerdings der Überzeung, dass es am besten ist für euch erst einmal die Grundlagen einer Sprache zu verstehen bevor ihr andere lernen solltet.

1.3 Installation

Bevor wir nun anfangen können uns mit **R** zu beschäftigen, müssen **R** und Rstudio erst einmal auf eurem Rechner installiert werden. Je nach dem Betriebssystem eurer Wahl (Linux, Windows, MacOS) findet ihr verschiedenen Abschnitte, welche euch den Ablauf der Installation verdeutlichen.

Der genrelle Ablauf ist wie folgt, zu erst installiert ihr **R** und dann erst Rstudio.

1.3.1 Installation unter Linux

Download des Base-**R** Installers über

- R Cran
- oder jeweiligen Package Manager

Download des von unser vorgeschlagenen IDEs Rstudios - Rstudio

Einige Linux-Distributionen führen oft nicht die aktuellste Version von R Studio in ihren Repos. Daher wäre es sinnvoll, die Version von R zu überprüfen und gegebenfalls die aktuellste Version über aktuelle Repos zu beziehen. Diese Aufgabe sollte für jeden Unix-Nutzer eine triviale Aufgabe sein.

1.3.2 Installation unter Windows:

Download des Base-**R** Installers unter

- R Cran
- Alternativ unter: GWDG

Download des von unser vorgeschlagenen IDEs Rstudios - Rstudio

1.3.3 Installation unter MacOS

Download des Base-**R** Installers wie bei Windows unter CRAN:

- R Cran
- Alternative über Homebrew

Download des von unser vorgeschlagenen IDEs Rstudios - Rstudio

Chapter 2

Interaktion mit R und RStudio

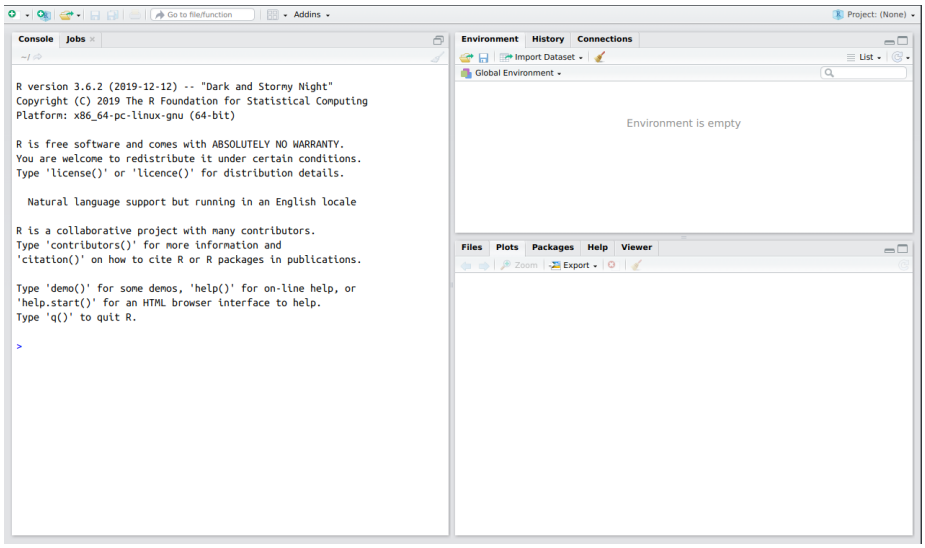
Du hast jetzt also **R** und **RStudio** auf deinem Rechner installiert und willst sofort loslegen, aber vorher ist es erst einmal wichtig zu verstehen, was der Unterschied zwischen **R** und **RStudio** ist und wieso du beide brauchst.

Während es sich bei **R** um eine Programmiersprache, also eine Sprache zur Interaktion mit einem Computersystem, handelt, handelt es sich bei **RStudio** um einen sogenannten IDE (integrated development environment). Ein IDE stellt Programmierern eine Sammlung der wichtigsten Werkzeuge zur Softwareentwicklung unter einer Oberfläche zur Verfügung.

Um nun mit und in **R** zu arbeiten rufst du also einfach **RStudio** auf. WICHTIG! Nicht **R** selbst aufrufen (auch wenn dieses funktioniert und einige alte hartgesottene Programmierer darauf schwören).

2.1 Das User Interface von RStudio

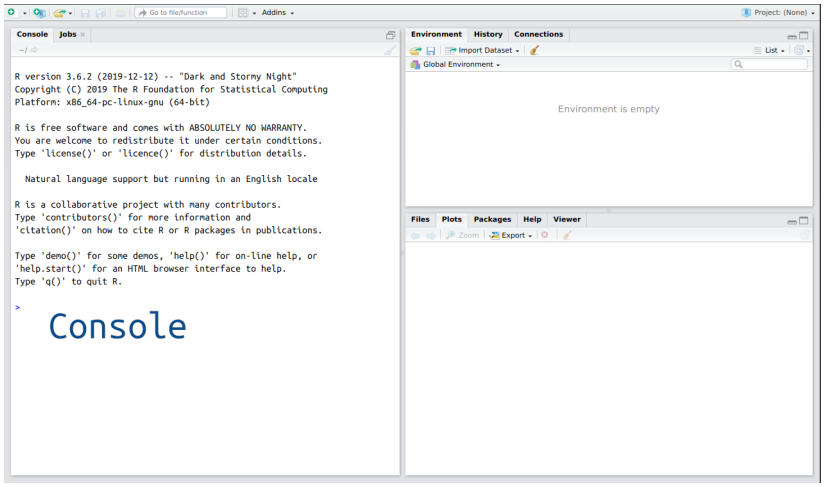
Wenn du **RStudio** zum ersten Mal öffnest wirst du von folgendem Bild begrüßt.



Was du hier siehst ist **RStudio** in seiner Standard Konfiguration. Im folgenden Abschnitt wirst du die drei bzw. vier Fenster oder auch Panes genannten Interaktionsfelder von **RStudio** näher kennen lernen und verstehen wie du diese benutzen kannst. Hierbei gehen wir von links nach rechts vor.

2.2 Die Console-Oberfläche

Das Console-Fenster findest du in der Grundausführung von **RStudio** auf der linken Seite und es dient dir als direktes Interaktionsfenster mit **R** und dadurch mit dem Computer. Jeder Code der in der Console eingegeben und durch Enter bestätigt wird, wird durchgeführt (engl. to run code).



Du kannst in die **R**-Console einfache Rechenaufgaben eingeben und **R** wird dir wie eine Art Taschenrechner die Lösung der Aufgabe ausgeben.

```
2 + 3
```

```
## [1] 5
```

Die **R**-Console ist allerdings viel mehr als eine Art Taschenrechner, du kannst hier dem Computer direkt über **R**-Code komplexe Befehle geben.

Es ist Zeit für deinen ersten selbst geschriebenen Code. Hierfür befehlst du dem Rechner die Worte "Hallo Welt!" widerzugeben. Dabei nutzt du den Befehl `print()`, gibst in die Klammer die Worte ein und führst den Code über das Drücken der Enter + Strg -Tasten aus (alternativ einfach auf den blauen Button "Run Code" klicken). Hierbei ist es wichtig die Worte in Anführungszeichen einzugeben `" "` um den Rechner zu symbolisieren, dass es sich um menschliche Sprache handelt.

```
print("Hallo Welt!")
```

```
## [1] "Hallo Welt!"
```

Herzlichen Glückwunsch! Du hast hiermit deinen ersten R-Code geschrieben.

Allerdings ist es oft schwer und sehr langwierig gewisse Befehle jedes Mal neu einzugeben, sodass **R** bereits eine Vielzahl an implementierten Befehlen wie den oben genutzten `print()` kennt. So kannst du ohne viel Programmierkenntnisse dem PC sagen Zahlen zu runden:

```
round(2.5)
```

```
## [1] 2
```

Oder die Uhrzeit deines Systems auszugeben:

```
Sys.time()
```

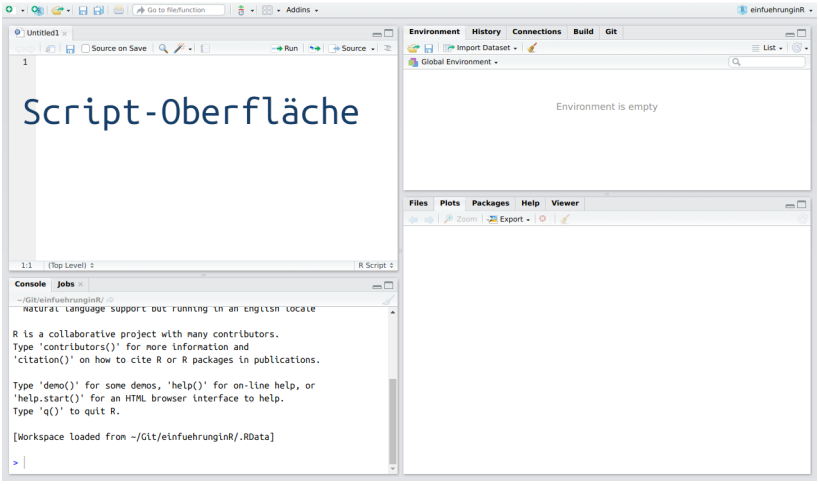
```
## [1] "2020-04-25 00:19:00 CEST"
```

Wie du mit den verschiedenen Funktionen richtig umgehst, neue hinzufügst und eigene selbst schreibst, wirst du im späteren Verlauf lernen können.

2.3 Die Script-Oberfläche

Das Eingeben von Code in die Console kann auf Dauer mühsam sein, vor allem wenn du denselben Code wiederholst, komplexere Eingaben tätigst oder aufeinander folgende Operationen ausführen willst. Daher kommen wir nun von der Console zur Script- oder Source-Oberfläche.

Diese Oberfläche ist beim ersten Starten von **RStudio** nicht zu sehen, kann allerdings über den Reiter File –> New File –> R Script geöffnet werden. Alternativ kannst du einfach ein neues Skript über die Tastenkombination: Strg + Shift + N öffnen. Hierbei sei die Nutzung des Keyboard-Shortcuts zu bevorzugen, da diese Art und Weise schneller und direkter von statten geht.



Wenn du nun Code in dein neues Skript eingibst, wirst du merken, dass dieser nicht sofort ausgeführt wird. Um den Code zu nutzen, musst du diesen an die Console "schicken". Hierfür gibt es verschiedene Methoden. Dabei kannst du zum einen den auszuführenden Code mit der Maus/Keyboard markieren und dann auf den Button "Run" klicken. Der viel einfachere und von uns vorgeschlagene Weg ist, einfach den Code durch das Drücken von Strg + Enter (Linux, Windows) bzw. Command + Enter (Mac) auszuführen.

Ein von dir geschriebenes Skript kannst du einfach über den Reiter File –> Save As abspeichern, sodass du das File jeder Zeit an dem Ort findest mit dem Namen "Euer_Skript_Name.R".

Shortcuts

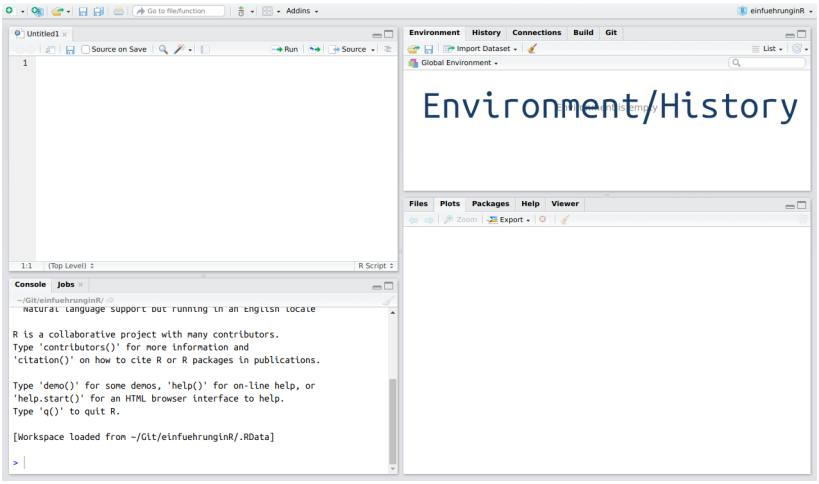
Es ist wichtig zu bedenken, dass Short-Cuts betriebssystemabhängig sind. Alle im Verlauf dieses Skripts genutzten Shortcuts beziehen sich auf die Linux/Windows Shortcuts. Wenn du daran interessiert bist zu sehen wie

weitere Shortcuts für Rstudio aussehen oder du ein Mac System benutzt kannst du einfach im Reiter „Tools“ eine Übersicht der Keyboard Shortcuts sehen.

Du wirst nun ein leeres Dokument sehen, in welchem du den R-Code schreiben kannst, ohne ihn gleich auszuführen. Diese Oberfläche bzw. das Skript selbst dient also als eine Art Sammelstelle für deinen Code. Das Skript kann ebenfalls gespeichert werden, sodass du in der Lage bist Code später (wieder) auszuführen oder ihn gar zu teilen oder später bei einer Seminar- oder Abschlussarbeit abzugeben.

2.4 Die Environment-Oberfläche

Das in der Ausgangskonfiguration sich rechts oben befindliche Environment-Feld dient verschiedener Aufgaben. In aller erster Linie sind dort alle Objekte zu finden welche gerade in R definiert sind.



Als kleines Beispiel der folgende Code:

```
1:10
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

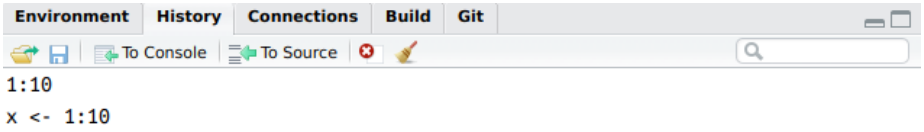
```
x <- 1:10
```

Hierbei erstellst du eine Sequenz an Zahlen von 1 bis 10. Im ersten Beispiel wird diese Sequenz ausgegeben, während beim zweiten Beispiel du einem Objekt `x` die Werte der Sequenz von 1 bis 10 zuweist. Im folgenden Kapitel werden wir im Genaueren die Thematik des Objektzuweisen behandeln. Für jetzt reicht es aus zu verstehen, dass durch die Zuweisung des Objektes der dabei zugewiesene

Wert gespeichert wird. Darüber hinaus erscheint nun rechts im Environment Feld der Eintrag für `x`. Dies bedeutet, wann immer du in der **R** Console oder im Skript die Größe `x` benutzt, die zugewiesene Sequenz von 1 bis 10 ausgegeben oder benutzt wird.



Wenn du auf den zweiten Tab „History“ klickst, wird dir eine chronologisch sortierte Auflistung aller von dir ausgeführtem **R** Befehle angezeigt. Dieser Tab hilft dir also als eine Art Anlaufstelle für Fehlersuche oder um zu verstehen, was du beim letzten Mal in **R** überhaupt gemacht hast.



Zu guter Letzt bietet die Environment-Oberfläche das manuelle Einlesen von Daten über den Butto “Import Dataset”. Indem du diesen Button bedienst, öffnet sich ein Dropdown Menü, in dem du die verschiedenen Arten von importierbaren Daten auswählen kannst. Dieses Menü erlaubt dir so das Einlesen und Bearbeiten von Daten aus

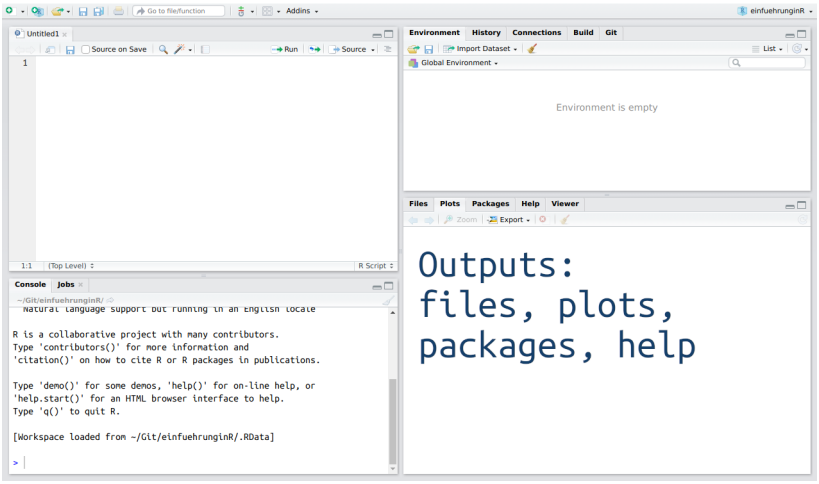
- Text-Datein (.txt, .csv, . . .)
- Excel (.xls, .xlsc, .xml, . . .)
- SPSS

- STATA
- SAS

Wie man nun diese Daten einließt, wird in einem der folgenden Kapiteln genauer beschrieben.

2.5 Die Output-Oberfläche

Die Output Oberfläche mit den vier Tabs Files / Plots / Packages / Help zeigt dir viele hilfreiche Informationen. Wir gehen jeden Tab im Detail durch.

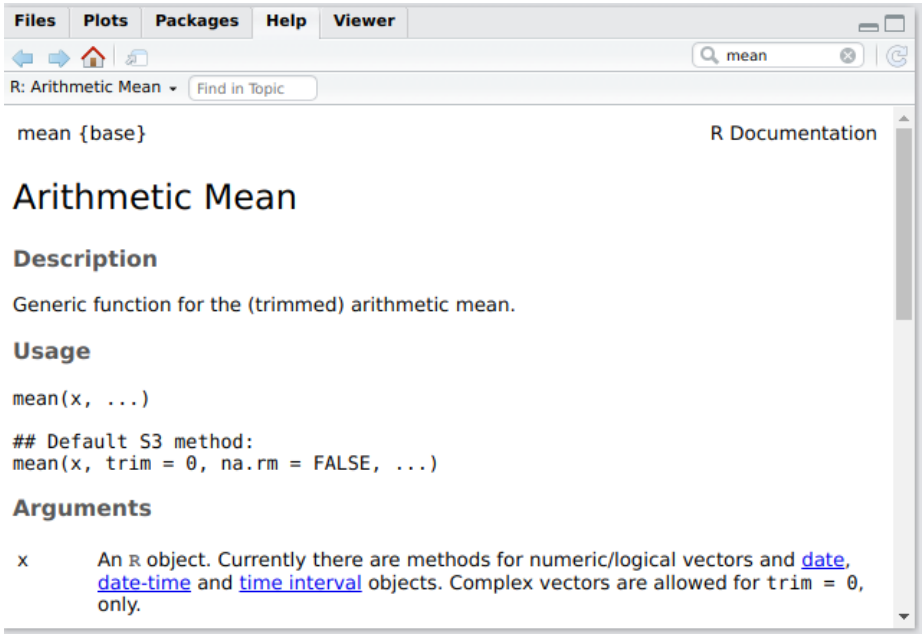


Files - Das Datei-Panel gibt dir Zugriff auf das Dateiverzeichnis auf deiner Festplatte. Eine nette Eigenschaft des "Dateien"-Panels ist, dass du es benutzen kannst, um dein Arbeitsverzeichnis festzulegen - sobald du zu einem Ordner navigierst, indem du Dateien lesen und speichern möchtest, klicke auf "Mehr" und dann auf "Als Arbeitsverzeichnis festlegen".

Plots - Das Plots-Panel zeigt alle deine Plots. Es gibt Schaltflächen zum Öffnen des Plots in einem separaten Fenster und zum Exportieren des Plots.

Packages - Zeigt eine Liste aller auf deiner Festplatte installierten **R**-Packages an ob sie derzeit geladen sind oder nicht. Packages, die in der aktuellen Sitzung geladen sind, werden geprüft, während installierte aber nicht geladene Packages nicht markiert sind.

Help - Hilfemenü für **R**-Funktionen. Du kannst entweder den Namen einer Funktion in das Suchfenster eingeben oder den Code verwenden, um nach einer Funktion mit dem Namen zu suchen.



Hierfür gibt es neben der Eingabe in das Suchfeld verschiedene weitere Möglichkeiten Hilfe zu finden. Die erste, du nutzt ein einziges `?` gefolgt von dem Namen der Funktion zu der du mehr wissen möchtest, oder du nutzt `??` gefolgt von einem Ausdruck zu dem du mehr wissen möchtest. Wie du hieran erkennen kannst, ist der Unterschied darin zu sehen, dass man nur ein Fragezeichen nutzt, wenn man bereits die Funktion kennt, um die es geht und zwei Fragezeichen, wenn man eine Funktion finden möchte, um ein Problem zu lösen.

```
?sd
# Oder auch möglich als
help(sd)
# Möchtest du eine Funktion für Quantile finden:
??quantile
```

Falls du nicht direkt die Lösung deines Problems findest, die Erklärung/Beispiel nicht verständlich genug oder dein Problem zu komplex für die interne **RStudio**-Hilfe ist, gibt es verschiedene Arten, um an Hilfe zu kommen.

Chapter 3

Objekte in R-Programmierung

Nach dem wir im letzten Kapitel uns **R** und dessen IDE RStudio genauer angeschaut und dabei erste Beispiele für das Programmieren in **R** gesehen haben, wird es Zeit uns in diesem Kapitel genauer mit der Programmierung in **R** zu befassen. Um zu verstehen, wie **R** funktioniert, musst du wissen, dass sich **R** um zwei Dinge dreht: Objekte und Funktionen. Fast alles in **R** ist entweder ein Objekt oder eine Funktion. Was ist ein Objekt? Ein Objekt ist ein Ding - wie eine Zahl, ein Datensatz, eine zusammenfassende Statistik wie ein Mittelwert oder eine Standardabweichung oder ein statistischer Test. Objekte gibt es in **R** in vielen verschiedenen Formen und Größen. Es gibt einfache Objekte wie einzelne Zahlen, Vektoren, die mehrere Zahlen repräsentieren, komplexere Objekte wie Datenrahmen, die Datentabellen darstellen, und noch komplexere Objekte wie Hypothesentests oder Regressionstests, die alle möglichen statistischen Informationen enthalten.

Nun was ist eine Funktion? Eine Funktion ist eine Prozedur, die typischerweise ein oder mehrere Objekte als Argumente (aka, Eingaben) nimmt, etwas mit diesen Objekten macht und dann ein neues Objekt zurückgibt.

3.1 Erstellen eines Objekts

Im letzten Kapitel haben wir ein Beispiel besprochen, indem wir einem Objekt `x` die Sequenz von 1 bis 10 zugewiesen haben. (`x <- 1:10`) Durch diese Zuweisung war die Sequenz an das Obkjekt `x` gebunden, sodass jedes Mal, wenn `x` in der Console abgerufen worden ist, die Sequenz von 1 bis 10 widergeben worden ist. Generell kann man **R** wie einen Taschenrechner nutzen, allerdings können wir ebenfalls wie bei dem Sequenz Beispiel Rechnungen, Werte oder andere Dinge

gewissen Ausdrücken wie dem Objekt `X` zuweisen um sie so zu speichern. Für einfache Zuweisungen, wie wir sie in diesem Kurs uns anschauen werden, nutzen wir immer den `<-` Operator. Also ein gewisser Wert, der rechts steht, wird einem Objekt das links steht zugewiesen. Du kannst auch `=` für Zuordnungen verwenden, aber nicht in jedem Kontext. Wegen der geringfügigen Unterschiede in der Syntax ist es sinnvoll, für Zuweisungen immer `<-` zu verwenden. Hierbei müssen Objekte nicht unbedingt einzelne Objekte wie `x` sein, sie können auch Namen tragen. Als Beispiel des Zuweisens von verschieden Werten zu verschiedenen Objekten.

```
x <- 13
x
```

```
## [1] 13
```

Hierbei wurde dem Objekt `x` der Wert 13 zugewiesen und im nächsten Schritt wird dem Objekt `x` erneut ein neuer Wert von 2 zugewiesen.

```
x <- 2
x
```

```
## [1] 2
```

Du kannst hierbei sehen, dass durch die zweite Zuweisung die erste Zuweisung aufgehoben und durch die neue ersetzt worden ist.

Solltest du deinem Code Kommentare hinzufügen wollen, welche nicht durch **R** ausgeführt werden sollen, nutzt du zuerst das Symbol `#`. Wobei es einfacher ist, erst deinen Kommentar zu formulieren, diesen zu markieren und durch das Drücken der Tastenkombination `Strg + Shift + c` wird dieser Part automatisch zu einem Kommentar.

```
# Kleines Beispiel eines String-Vectors

beispiel <- "hallo welt!"
beispiel
```

```
## [1] "hallo welt!"
```

Das dritte Beispiel zeigt dir, dass ebenfalls ganze Namen oder andere Zeichenkombinationen einen Wert zugewiesen bekommen können. Dir fällt ebenfalls auf, dass das dritte Beispiel keinen Zahlenwert darstellt, sondern Text, einen sogenannten String. Zu den verschiedenen Datentypen in **R** kommen wir im nächsten Kapitel

3.2 Namen von Objekten

Gute Objektnamen schaffen ein Gleichgewicht zwischen einfacher Eingabe (d.h. kurze Namen) und leichter Interpretation. Wenn du mehrere Datensätze hast, ist es wahrscheinlich keine gute Idee, sie a, b, c zu nennen, weil du dann vergisst, welcher welcher ist. Wenn du jedoch lange Namen wie `NeuerDatensatz2020ErsteWocheApril` verwendest, bekommst du das Karpaltunnelsyndrom.

R unterscheidet übrigens zwischen Groß- und Kleinschreibung bei Objektnamen. Wenn wir ein Obejekt `alleTeilnehmer` definiert hätten, können wir nicht mit `AlleTeilnehmer` weiterrechnen.

Es gibt allerdings einige Namen für Objekte welche du nicht in **R** nutzen solltest, da diese zu Problem führen (können). Die Namen der Objekte sollten die folgenden Zeichen nicht enthalten `^, !, $, @, +, -, /, or *`.

Als letzte Regel soll man keine bereits bestehenden Funktionsnamen überschreiben, wie z.B. `mean` oder `lm`.

3.3 Verändern von Objekten

Wir haben bereits gesehen, dass wenn man einem Objekt wiederholt einen neuen Wert zuweist, dann behält das Objekt den neusten Wert. Was aber nun, wenn du zu dem bestehenden Wert deines Objekts `x <- 4` weitere `4` dazu addieren willst?

Um dies zu tun, musst du einfach `x + 4` berechnen

```
x <- 4
x + 4
```

```
## [1] 8
```

Wenn du dir allerdings den Wert von `x` nun ansiehst, wirst du feststellen, dass sich der Wert nicht verändert hat.

```
x
```

```
## [1] 4
```

Um nun bereits zugewiesene Objekte zu verändern, musst du es erneut zuweisen. Egal, was du mit einem Objekt machst, wenn du es nicht erneut zuweist, wird es sich nicht ändern. Das Problem hierbei ist, dass **R** nur verstanden hat, dass es ausgeben soll was `x + 4` ergibt. Um nun `x` den Wert von `x + 4` zu zuweisen, musst du `x` den Wert von `x + 4` zuweisen. Dies erfolgt so

```r
x <- 4
x <- x + 4
x
```

```
## [1] 8
```

Chapter 4

Datenstrukturen in R

Um nun mit **R** zu arbeiten, ist es wichtig zu verstehen wie **R** überhaupt mit Daten umgeht, sie speichert und viel mehr noch, wie du als Nutzer mit diesen arbeiten kannst. Daher schauen wir uns zu Beginn einmal an, in welchen Formen (Strukturen) Daten in **R** gespeichert werden können.

Die Basisdatenstrukturen von **R** können nach ihrer Dimensionalität (1 x d, 2 x d oder n x d) und danach kategorisiert werden, ob sie homogen (alle Inhalte müssen vom gleichen Typ sein) oder heterogenen (die Inhalte können von unterschiedlichem Typ sein) sind. Daraus ergeben sich die fünf in der Datenanalyse am häufigsten verwendeten Datentypen:

Dimension	Homogen	Heterogen
1 x d	(Atomarer) Vektor	List
2 x d	Matrix	Data Frame
n x d	Array	-

Eine Besonderheit von **R** ist es, dass es keine skalaren Typen kennt. Einzelne Daten, die wir als Menschen als Skalare wahrnehmen würden, sind trotzdem für **R** ein Vektor mit der Länge 1.

Wir starten mit dem einfachsten und wahrscheinlich wichtigsten Data Type in **R** den Vektoren. Ein Vektor besteht entweder homogen aus der gleichen elementaren oder "atomaren" Klasse (sog. Atomare Vektoren) oder aus heterogen Daten als Listen.

4.1 Vektoren

Um atomare Vektoren zu erstellen, nutzt man den `c()`-Operator, der kurz für combine steht. Hierfür weisen wir einfach einem Objekt einen Vektor dadurch zu, dass wir in die Klammern des `c()`-Operators die jeweiligen homogenen atomaren Objekte aufführen, die indem Vektor vorhanden sein sollen.

```
x <- c(1, 2, 3)
```

Nun stellt sich allerdings erst einmal die Frage, was diese atomaren Objekte sind? Hierfür gehen wir in diesem Skript die für uns wichtigsten vier sogenannter atomaren (also kleinst mögliche Form) Objekten in **R** durch. Jede der verschiedenen Formen verlangt eine andere Form der Notation. Hierbei handelt es sich um die

1. Logical
2. Integer
3. Double
4. Character

4.1.1 Logicals

Bei den **Logicals** handelt es sich, wie der Name schon verrät, um logische Aussagen. Logische Werte nehmen einen von zwei möglichen Werten an `TRUE` oder `FALSE`, bzw. deren Abkürzung in Form von `T` und `F`.

```
logicalBSP <- c("FALSE", "TRUE", T, F)
```

4.1.2 Doubles

Bei den sogenannten **Doubles** handelt es sich um Gleitkommazahlen. Hierbei können die Werte in Decimal-, Scientific- und in Hexadecimalschreibweise angegeben werden. Es gibt außerdem noch besondere Zahlenwerte, welche ebenfalls zu den Doubles gehören. Hierbei handelt es sich um Unendlich: `Inf` bzw. `-Inf` und um die Angabe "Not a Number" `NaN`.

```
doublesBSP <- c(42.42, 13.37, 6.66)
```

4.1.3 Integers

Die **Integers** (oder auch ganzzahlige Zahlen) gehören genau wie die Doubles zu den Zahlenwerten, die Besonderheit in der Notation liegt darin, dass jedem Zahlenwert ein "L" angehängt wird (`1L`)

```
integerBSP <- c(2L,6L,1L,3L,3L)
```

4.1.4 Characters

Die Klasse der **Characters** umfasst alle Eingaben von Buchstaben/Zeichenkombinationen. Bei der Eingabe werden diese immer mit Anführungsstrichen (" " oder ' ') umschlossen. Beispiel: `"Hallo Welt!"`

```
stringBSP <- c("Hallo Welt!", "Ich bin R Code!")
```

Korrekterweise sei an dieser Stelle auf zwei weitere atomare Objektarten hingewiesen, die complex- und raw-Objekte. Die Klasse "complex" umfasst alle Eingaben von komplexen Zahlen, während die Klasse "raw" nur im Rahmen von binären Daten genutzt wird. Diese beiden Arten werden in komplexeren Zusammenhängen behandelt, welche dir wahrscheinlich nicht im Rahmen der Bachelorveranstaltungen über den Weg laufen und werden daher nicht behandelt. Falls du Interesse an diesem Thema hast empfehlen wir das Buch: Advanced R von Wickham

4.2 Listen

Der große Unterschied zwischen atomaren Vektoren und Listen ist der, dass Listen alle möglichen verschiedenen Typen von Daten beinhalten können. Listen werden im Gegensatz zu Vektoren mit dem Befehl `list()` erstellt.

```
lst1 <- list(3L, "Hallo", c(13.9, 42.22223), c(T, F, TRUE))
```

Interessanterweise ist es mögliche Listen in Listen zu speichern, was es ermöglicht sogenannte rekursive Vektoren zu erstellen:

```
listception <- list(list(list(list())))
```

4.2.1 Eingabe von Vektoren und Listen

Die Eingabe von Daten per Hand kann sehr langwierig sein, vor allem wenn du zum Beispiel die Folge von 1 bis 1000 als Vektor speichern willst. Daher gibt es Möglichkeiten Einträge über Befehle von **R** automatisiert zu erstellen. Hier sind ein paar der nützlichsten Methoden, die dir das Arbeiten mit **R** erleichtern:

Funktion	Beispiel	Ergebnis
`c(a, b, ...)`	`c(1, 5, 9)`	1, 5, 9
`a:b`	`1:5`	1, 2, 3, 4, 5
`seq(from, to, by, length.out)`	`seq(from = 0, to = 6, by = 2)`	0, 2, 4, 6
`rep(x, times, each, length.out)`	`rep(c(7, 8), times = 2, each = 2)`	7, 7, 8, 8, 7, 7, 8, 8

4.2.2 Faktoren

Eine spezielle Form der Vektoren sind Faktoren, welche die Kategorien oder Klassifikationen von Vektoren angeben, welche also Kategoriale Daten darstellen. Durch die Anwendung der `factor()`-Funktion auf einen Vektor mit Characters bzw. Zeichenketten (Strings), verwandelt diese einfachen die Zeichen in Faktoren um. Als Beispiel erschaffen wir einen Vektor mit Bezeichnung für Frau (w) und Mann (m) und verwandeln diese in Faktoren um.

```
sex <- rep(c("w","m"), times = 3)
sex <- factor(sex)
sex
```

```
## [1] w m w m w m
## Levels: m w
```

Zwei nützliche Funktionen bei der Arbeit mit Faktoren sind die Checks `class()` und `levels()`. Die erste Funktion fragt einen Vektor, welche Art von Daten vorliegen, während die zweite Funktion die verschiedenen Ausprägungen des Faktors darstellt.

```
class(sex)
```

```
## [1] "factor"
```

```
levels(sex)
```

```
## [1] "m" "w"
```

4.3 Matrizen und Arrays

Atomare Vektoren und Listen sind, wie oben bereits erwähnt, als 1 x d Objekte zu verstehen. Fügen wir den Objekten nun eine weitere Dimension hinzu, so betrachten wir Matrix und Arrays. Ähnlich zu den Listen existieren zur Erstellung von Matrizen und Arrays eigene Befehle, `matrix()` und respektive `array()`.

4.3.1 Matrizen

Das besondere bei Matrizen im Gegensatz zu den 1-dimensionalen Vektoren und Listen ist, dass sie mehrere Spezifikationen benötigen, um erschaffen zu werden. So muss bei der Nutzung von `matrix()` definiert werden, welche Werte in der Matrix zu finden sein sollen, des Weiteren wie die Dimensionen der Matrix auszusehen haben, also die Anzahl der Reihen (`nrow`) und Spalten(`ncol`). Hier ein kleines Beispiel:

```
m <- matrix(c(1,2,3,4,5,6), ncol = 2, nrow = 3)
m
```

```
##      [,1] [,2]
## [1,]    1    4
## [2,]    2    5
## [3,]    3    6
```

Du kannst sehen, dass zuerst ein Atomarvektor mithilfe des Combine-Befehls eingegeben wird, gefolgt von der Anzahl der Spalten mit 2 und der Anzahl der Reihen mit 3.

4.3.2 Arrays

Der Vollständigkeit halber hier ein Beispiel wie man Arrays in **R** erschafft, allerdings werden keine Arrays in der Einführungs-Vorlesung der Statistik behandelt.

Ähnlich zu der Logik hinter dem Matrix Befehl müssen mehr Eingaben getätigt werden, als nur die Datenpunkte selbst, zu sehen am folgenden Beispiel

```
a <- array(1:12, c(2, 3, 2))
a
```

```
## , , 1
##
##      [,1] [,2] [,3]
```

```
## [1,]    1    3    5
## [2,]    2    4    6
##
## , , 2
##
##      [,1] [,2] [,3]
## [1,]    7    9   11
## [2,]    8   10   12
```

Der Array Befehl folgt folgender Struktur, man definiert zum einen die abzubildenden Datenpunkte, hier in unserem Beispiel durch die Sequenz von 1 bis 12 `1:12`, gefolgt von der Eingabe der gewünschten Dimension, in diesem Beispiel durch den Vektor `c(2,3,2)`, resultierend in einem Array mit den Dimensionen `2 x 3 x 2`.

4.3.3 Data.Frames

Data Frames sind die zweidimensionale Version einer Liste. Sie sind sehr flexibel und eine nützliche Datenstruktur für die Datenanalyse. Man kann einen Data Frame als eine Art **R** Äquivalent zur Excel-Tabelle verstehen, da sie Daten in einem ähnlichen Format speichert und darstellt.

Data Frames gruppiert Vektoren in einer zweidimensionalen Tabelle. Jeder Vektor wird zu einer Spalte in der Tabelle. Folglich kann jede Spalte eines Datenrahmens einen anderen Datentyp enthalten.

Die Erstellung eines Data Frames von Hand erfordert viel Arbeit, sodass die Eingabe einfacher über die `data.frame()` Funktion geschieht. Nutze einfach eine beliebige Anzahl von Vektoren, die jeweils durch ein Komma getrennt werden. Jeder Vektor sollte einem Namen entsprechen, der den Vektor beschreibt. Diese Vektoren stellen dann die Variablen in dem Data Frame dar Wichtig, jeder Vektor muss hierbei die gleiche Länge haben. Die `data.frame()`-Funktion macht jeden Vektor zu einer Spalte des neuen Data Frames:

```
df <- data.frame(lat = c("51.5454", "53.1438", "35.7034"),
                 lon = c("9.9055", "8.2138", "139.7532"),
                 name = c("Göttingen", "Oldenburg", "Tokyo"))
df
```

```
##       lat      lon      name
## 1 51.5454   9.9055 Göttingen
## 2 53.1438   8.2138 Oldenburg
## 3 35.7034 139.7532     Tokyo
```

Die Variablen in diesem Data Frame sind Breitengrad (`lat`), Längengrad (`lon`) und Name (`name`).

4.3.4 Names

Es macht oft Sinn nicht nur den Objekten, sondern auch Variablen in den **R**-Objekten Namen zu geben. Ein Beispiel:

```
alter <- c(13, 42, 39)
names(alter)
```

```
## NULL
```

```
names(alter) <- c("Chris", "Henny", "Paul")
alter
```

```
## Chris Henny  Paul
##    13    42    39
```

Wir haben hier also einen Vektor der verschiedene Altersangabe umfasst. Wir können nun jedem Eintrag einen Namen mit der `name()`-Funktion zuweisen, in diesem Fall drei verschiedene Namen. Wenn nun der Vektor abgerufen wird, werden die Namen mit den dazugehörigen Alterszahlen angegeben. Listen und Matrizen können genauso wie atomare Vektoren Namen aufweisen. Bei Listen ist dies sehr hilfreich, da es so einfacher ist auf bestimmte Elemente in den Listen zuzugreifen.

```
alter <- list("Chris" = 13, Henny = 42, Paul = 39)
alter
```

```
## $Chris
## [1] 13
##
## $Henny
## [1] 42
##
## $Paul
## [1] 39
```

Wie man hier sehen kann, enthält die Liste nun drei verschiedene Werte mit jeweils einem anderen Namen. Interessanterweise kann man den Namen mit aber auch ohne Anführungszeichen eingeben, dies liegt daran, dass wenn der Name aus mehreren Worten besteht der Name von `" "` umschlossen sein muss. Diese Bedingung ist aber bei einem einzigen Wort optional. Des Weiteren sind die Namen hilfreich, um auf einzelne Elemente der Liste zuzugreifen.

```
alter$Chris
```

```
## [1] 13
```

Aber zu diesem bestimmten auswählen von Teildaten und Subsets mehr im späteren Abschnitt zur Daten Manipulation. Namen bei Matrizen haben auf Grund ihrer anderen Dimensionalität eine andere Struktur. Matrizen haben immer Column (Spalten) und Row (Reihen) spezifische Namen. Diese Namen kann man nur über die Nutzung der Funktionen `colnames()` und `rownames()` festlegen und verändern oder indem man über den Befehl `dimnames()` gleich beide zusammen definiert.

```
m <- matrix(1:4, nrow = 2, ncol = 2)
dimnames(m) <- list(c("a", "b"), c("c", "d"))
m
```

```
##   c d
## a 1 3
## b 2 4
```

```
colnames(m) <- c("h", "f")
rownames(m) <- c("x", "z")
m
```

```
##   h f
## x 1 3
## z 2 4
```

Chapter 5

Grundlegende R-Funktionen

R kommt von Haus aus mit verschiedenen bereits implementierten Funktionen, wie die bereits vorgestellten `mean()`, `round()`, `print()` und vielen mehr. Im folgenden Abschnitt werden wir durch ein paar der wichtigsten in **R** implementierten Funktionen durchgehen, um dir ein Gefühl dafür zu geben, wie du mit **R**-Funktionen umgehen musst und wie du effektiv mit **R** arbeiten kannst.

5.1 Vektorisierte mathematische Operationen

Bisher weißt du, wie man grundlegende arithmetische Operationen wie `+` (Addition), `-` (Subtraktion) und `*` (Multiplikation) auf Skalaren ausführst. Glücklicherweise macht es **R** genauso einfach, arithmetische Operationen mit numerischen Vektoren durchzuführen:

```
x <- c(1, 2, 3, 4, 5)
y <- c(5, 6, 7, 8, 9)
```

Wenn du eine Operation mit einem Vektor und einem Skalar durchführst, wendet **R** den Skalar auf jedes einzelne Element des Vektors an.

```
# Addition mit Skalaren
x + 100
```

```
## [1] 101 102 103 104 105
```

```
1:5 + 10
```

```
## [1] 11 12 13 14 15
```

Wenn du eine Operation auf zwei Vektoren der gleichen Länge anwenden willst funktioniert es recht einfach. **R** wendet die Operation auf beide Vektoren Element für Element an. Als Beispiel was damit gemeint ist:

```
x + y
```

```
## [1]  6  8 10 12 14
```

```
y - x
```

```
## [1] 4 4 4 4 4
```

```
(x + y) / 10
```

```
## [1] 0.6 0.8 1.0 1.2 1.4
```

5.2 Vektorisierte Matrix-Operationen

Matrix-Operationen sind ebenfalls vektorisiert, was eine schöne kompakte Notation ermöglicht. Auf diese Weise können wir Element für Element Operationen auf Matrizen durchführen, ohne jedes Element in einer Schleife zu wiederholen.

Um zu verdeutlichen, wie Matrix-Operationen funktionieren, erstellen wir einfach zwei Matrizen `m1` und `m2`.

```
m1 <- matrix(1:4, 2, 2)
m2 <- matrix(rep(10, 4), 2, 2)
```

Operationen wie Multiplikation oder Division erfolgen bei Matrizen auf zwei verschiedene Arten.

Für die "klassische" Multiplikation von Matrizen in **R** wird der **%*%**-Operator genutzt. Um unsere beiden Matrizen zu multiplizieren gehst du einfach wie folgt vor

```
m1 %*% m2
```

```
##      [,1] [,2]
## [1,]   40   40
## [2,]   60   60
```

Für ein elementweises Produkt (Hadamard-Produkt) verwendest du den einfachen Multiplikations-Operator `*`. Angewandt auf unser Beispiel, ergibt sich der folgende Code,

```
m1 * m2
```

```
##      [,1] [,2]
## [1,]   10   30
## [2,]   20   40
```

Für eine elementweise Division gehen wir analog vor,

```
m1 / m2
```

```
##      [,1] [,2]
## [1,]  0.1  0.3
## [2,]  0.2  0.4
```

5.3 R Operatoren

R hat viele Operatoren, um verschiedene mathematische und logische Operationen durchzuführen. Einige hast du bereits im Verlauf dieses Skriptes kennengelernt. Das folgende Kapitel wird dir eine Zusammenfassung der wichtigsten **R** Operatoren geben.

5.3.1 Arithmetische Operatoren

Diese Operatoren werden verwendet, um mathematische Operationen wie Addition und Multiplikation durchzuführen. Hier ist eine Liste der in **R** verfügbaren arithmetischen Operatoren

Ausdruck	Beschreibung
`+`	Addition
`-`	Subtraktion
`*`	Multiplikation
`/`	Division
`^`	Exponent
`%%`	Modulus
`%/%`	Integer Division

5.3.2 Relationale Operatoren

Relationale Operatoren werden zum Vergleich zwischen Werten/objekten verwendet. Hier ist eine Liste der in **R** verfügbaren relationalen Operatoren.

Ausdruck	Beschreibung
==	Gleich
!=	Ungleich
<, >	KLeiner, Größer
<=	Kleiner gleich
>=	Größer gleich

Möchtest du z.B. herausfinden inwiefern zwei Objekte zu einander in Beziehung stehen, kannst du diese Operatoren nutzen. Als Beispiel sei gegeben

```
x <- 3
y <- 7

x < y
```

```
## [1] TRUE
```

```
x == y
```

```
## [1] FALSE
```

Die oben genannten Operatoren funktionieren mit Vektoren. Alle Operationen werden elementweise durchgeführt. Hier ist ein Beispiel.

```
x <- c(2,8,3)
y <- c(6,4,1)

x < y
```

```
## [1]  TRUE FALSE FALSE
```

```
x == y
```

```
## [1] FALSE FALSE FALSE
```

5.3.3 Logische Operatoren

Logische Operatoren werden verwendet, um boolesche Operationen wie `AND`, `OR` usw. auszuführen. Die Operatoren `&` und `|` führen eine elementweise Operation durch, die ein Ergebnis mit der Länge des längeren Operanden ergibt.

Aber `&&` und `||` untersuchen nur das erste Element der Operanden, die zu einem logischen Vektor mit einer einzigen Länge führen.

Null wird als `FALSE` betrachtet und Nicht-Null-Zahlen werden als `TRUE` genommen.

Ausdruck	Beschreibung
`!`	Logisches NICHT
`&`	Elementweises logisches UND
`&&`	Logisches UND
`|`	Elementweises logisches ODER
`||`	Logisches ODER

5.3.4 Deskriptive Statistik

Die folgenden Befehle stellen eine Zusammenfassung der für dich im Laufe des Modules wichtigsten beschreibenden Größen der Statistik.

Funktion	Beispiel	Output
`sum(x), product(x)`	`sum(1:10)`	55
`min(x), max(x)`	`min(1:10)`	1
`mean(x), median(x)`	`mean(1:10)`	5.5
`sd(x), var(x), range(x)`	`sd(1:10)`	3.0276504
`quantile(x, probs)`	`quantile(1:10, probs = .2)`	2.8
`summary(x)`	`summary(1:10)`	`Min = 1.00. 1st Qu. = 3.25, Median = 5.50, Mean = 5.50, 3rd Qu. = 7.75, Max = 10.0`

5.4 Missing Values

Ein Problem, welches auftreten kann, wenn man mit echten Daten arbeitet, ist, dass entweder falsche oder gar fehlende Werte auftreten. Diese fehlenden Werte kennt **R** als NAs (not available). Sollten in deinen Daten fehlende Werte auftreten, kommen leider viele grundlegende Statistiken zu nicht verlässlichen Ergebnissen. Der folgende Code gibt beispielsweise NA als Ergebnis zurück, weil ein NA-Wert im Datenvektor vorhanden ist:

```
a <- c(0, 1, NA, 3, 4)
mean(a)
```

```
## [1] NA
```

Glücklicherweise bietet **R** allerdings eine Möglichkeit, um mit `NAs` in den Daten umzugehen. So kann man entweder bei der Anwendung einer Funktion auf die Daten ein weiteres Argument an die Funktion weitergeben, welche dieser befiehlt, alle `NAs` in den Daten zu ignorieren. Bei diesem Argument handelt es sich um den Ausdruck `na.rm`, wenn dieses gleich `TRUE` gesetzt wird, ignoriert **R** alle `NAs`. Hier ein Beispiel:

```
mean(a, na.rm = TRUE)
```

```
## [1] 2
```

Oder es gibt die Möglichkeit alle `NAs` in den Daten zu löschen. Hierfür erstellt man einfach ein neues **R**-Objekt, wobei man die **R**-Funktion `na.omit()` auf die Daten anwendet. Als Beispiel:

```
b <- na.omit(a)
mean(b)
```

```
## [1] 2
```

5.5 Packages

Sollte die in der base-**R** Version enthaltenen Funktionen nicht für deine Arbeit ausreichen, gibt es zwei Möglichkeiten wie du mit diesem Problem umgehst:

1. Finden eines **R**-Packages, welches **R** um Funktionen erweitert, welche es dir erlauben dein Problem zu lösen

2. Eigene Funktionen schreiben.

Ein Package ist eine Sammlung von Code, welches von einer anderen Person oder Gruppe geschrieben worden ist. Meistens sind Packages dazu gedacht, ein bestimmtes Problem zu lösen, also Funktionen zusammenzufassen, die mit einem bestimmten datenwissenschaftlichen Problem zusammenhängen (z.B. Data Wrangling, Visualisierung, Inferenz). Jeder kann ein Package schreiben, und man kann Packages von vielen verschiedenen Orten bekommen.

5.5.1 Installation von Packages

Die Installation eines Packages bedeutet einfach das Herunterladen des Packages auf deinen Computer. Es gibt verschiedene Wege neue Packages zu installieren. Der einfachste und gebräuchlichste Weg ist das Herunterladen aus dem Comprehensive **R** Archive Network (CRAN). CRAN ist der zentrale Speicherort für **R**-Packages. Um ein neues **R**-Package von CRAN zu installieren, kannst du einfach den Befehl `install.packages("name")` ausführen, wobei "name" der Name des Packages ist.

```
install.packages("name")
```

Einige Packages sind Bündel von Packages. Zum Beispiel das "tidyverse"-Package, welches viele einzelne Packages für Daten Wrangling und Visualisierung zusammenfasst, so dass du bei der Installation von einem einzelnen Package des tidyverse eigentlich acht verschiedene Packages installierst.

```
# Installation des Package Bundles tidyverse
  install.packages("tidyverse")
```

5.5.2 Wie nutze ich Packages?

Jetzt weißt du wie man Packages installiert, aber wie nutzt du diese? Um Packages zu nutzen, welche nicht Teil der Base-R Version sind, müssen diese erst in die aktive **R** Session geladen werden. Hierfür nutzt du einfach den Befehl `library("name")`. Nehmen wir das Beispiel des Packages `ggplot2`, welches Teil des tidyverse ist. Dazu nutzt du einfach den folgenden Code:

```
# Laden des Packages ggplot2
  library(ggplot2)
```

Und schon bist du in der Lage die im Package beinhalteten Funktionen zu nutzen, um z.B. durch `ggplot2` ansprechendere graphische Analysen anzufertigen:

<img src="buch_files/figure-latex/ggplotBSP-1.pdf"">

Chapter 6

Funktionen und Kontroll-Strukturen

Bei einigen Aufgaben müssen sich Teile einer Berechnung wiederholen oder sie sind abhängig von bestimmten externen Bedingungen. Selbst denselben Befehl immer wieder abzutippen ist im besten Fall langweilig und im schlimmsten Fall sehr aufwendig. Deswegen erlaubt es **R**, wie viele andere Programmiersprachen auch, repetitive Aufgaben zu automatisieren.

6.1 For-Loops

Wenn du Rechnungen mehrfach wiederholen musst eignet sich dafür der `for()`-Befehl, der die folgende allgemeine Form besitzt:

```
for (variable in vector) {
  # Aufgabe
}
```

`variable` ist eine sogenannte Zählvariable, sie "zählt" wie viele Durchgänge der For-loop bereits hinter sich hat. Der `vector` ist die Spannweite von der die Zählvariable zählt, bis der For-Loop abgeschlossen ist. Dies erscheint vielleicht relativ abstrakt, aber das folgende Beispiel sollte verdeutlichen wie ein For-Loop funktioniert.

So möchtest du zum Beispiel die Zahlen von 1 bis 5 in der **R** Console ausgeben, wird diese Aufgabe mithilfe des For-Loops sehr einfach. Dafür musst du einfach nur den folgenden Code tippen:

```r
for(i in 1:5){
  print(i)
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```

Die Variable `i` nimmt je nach Durchlauf des Loops einen Zahlenwert von 1 bis 5 an. In jedem Durchlauf wird der jeweilige Wert von `i` ausgegeben. Dieser Vorgang startet bei 1 und endet mit dem 5ten Durchlauf.

6.2 If-Else

Du wirst beim Schreiben von eigenen Funktionen in die Situation kommen, in der eine Funktion bedingt auf ein Ereignis reagieren muss. Diese bedingten Abfragen sehen in **R** so aus

```r
# Einfacher Fall
  if (condition) {
# expr
}

# Entweder Oder Fall
if (condition) {
  # expr
} else {
  # expr
}
```

Der Ablauf hierbei ist immer gleich, wird eine bestimmte Bedingung (condition) erfüllt, wird ein bestimmter **R** Befehl ausgeführt (expr). Man kann diese If-Abfrage durch eine Else-Condition erweitern, welche bei Nichterfüllung der Bedingung einen anderen Befehl ausführt.

```r
x <- c(6)

if (x == 6) { # Wenn x = 6, dann
  print("x gleich 6")
}
```

```
## [1] "x gleich 6"
```

```
if (x == 5) { # Wenn x = 5, dann
  print("x gleich 5")
} else { # Wenn x nicht gleich 5 dann
  print("x ist ungleich 5")
}
```

```
## [1] "x ist ungleich 5"
```

6.3 While-Loops

Die while-Schleife dient dazu, eine Abfolge von Anweisungen mehrfach auszuführen, solange eine bestimmte Bedingung erfüllt ist.

```
while (condition) {
  # Expr
}
```

Ein Beispiel für diese Form der wiederholten Ausführung eines Befehls ist hier zu sehen. So lange der Wert der Variable `i` kleiner als 6 ist, wird der jeweilige Wert von `i` ausgegeben.

```
i <- 1
while (i < 6) {
  print(i)
  i = i+1
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
```

6.4 Eigene Funktionen

Wenn du merkst, dass du einige Aufgaben immer wieder durchführen musst oder dass keine Funktionen existieren, welche eine bestimmte Aufgabe lösen können, solltest du überlegen eigene Funktionen zu schreiben. Funktionen werden mit der `function()`-Anweisung definiert und wie alles andere in **R** als Objekte gespeichert. Ein Beispiel einer "leeren" Funktion

```r
f <- function() {
  # Eine leere Funktion
  }
```

Als nächstes können wir eine Funktion erstellen, die tatsächlich einen nicht-trivialen Funktionskörper hat. Das bedeutet, dass die Funktion auch eine wirkliche Aufgabe hat und etwas für dich ausführt. In unserem Beispiel ist es die Aufgabe der Funktion bei jedem Aufrufen der Funktion die Worte „Hello World!“ auszugeben.

```r
f <- function() {
  cat("Hello, world!\n")
}
f()
```

```
## Hello, world!
```

Der letzte Aspekt einer Grundfunktion sind die Funktionsargumente. Dies sind die Argumente, die der Benutzer angeben kann, auf deren Basis bestimmte Aktionen durchgeführt werden. Für unsere Grundfunktion können wir ein Argument hinzufügen, das bestimmt wie oft “Hallo, Welt!” in der Konsole ausgegeben wird.

```r
f <- function(num) {
       for(i in seq_len(num)) {
               cat("Hello, world!\n")
       }
}

f(2)
```

```
## Hello, world!
## Hello, world!
```

Zu guter Letzt lernst du noch den Befehl `return()` kennen. Dieser Befehl sagt deiner **R**-Funktion, dass, wenn die Funktion durchgeführt wird, es möglich ist einem Objekt mit Hilfe der Funktion einen Wert zuzuweisen. Als Beispiel, schreiben wir eine Funktion, welche zwei Elemente als Input nimmt und diese miteinander addiert.

```r
addtwo <- function(x, y){
  x + y
}
```

Führen wir nun diese Funktion aus, so werden `x` und `y` addiert und ausgegeben. Willst du nun aber zum Beispiel diesen Wert in einem Objekt speichern, nennen wir es einfach `v`, hierfür nutzt man einfach die folgende Funktionsstruktur.

```
addtwo <- function(x, y){
  z <- x + y
  return(z)
}
```

Gebe einfach beliebige Zahlen für die beiden Inputs ein. Du wirst sehen, dass kein Ergebnis ausgegeben wird, allerdings wenn du `v` eingibst und über `Strg + Enter` abrufst das Ergebnis ausgegeben wird.

```
addtwo <- function(x, y){
  z <- x + y
  return(z)
}

v <- addtwo(x = 2, y = 3)

v
```

Chapter 7

Daten

In diesem Kapitel wirst du lernen wie man externe Daten in **R** einließt, wie man mit diesen Daten arbeitet, sie verändert, anpasst und exportiert.

7.1 Einlesen von Daten

Daten können aus verschiedenen externen Quellen und Dateiformaten in **R** eingelesen werden. Es gibt hierfür ein paar Hauptfunktionen:

- `read.table()`, `read.csv()`, zum Einlesen von Tabellendaten
- `source()`, zum Einlesen von **R**-Code-Dateien
- `dget()`, zum Einlesen von **R**-Code-Dateien
- `load()`, zum Lesen in gespeicherten Arbeitsbereichen
- `unserialize()`, zum Lesen einzelner R-Objekte in binärer Form

Hierbei sei darauf hingewiesen, dass es noch viele weitere **R** Packages gibt, welche es dir erlauben Daten in **R** einzulesen. Parallel zu den verschiedenen Funktionen zum Einlesen von Daten in **R** gibt es Funktionen, welche es dir erlauben Daten zu schreiben/exportieren.

- `write.table()`, `write.csv()` zum Schreiben von Tabellendaten in Textdateien (z.B. CSV)
- `writeLines()`, zum zeilenweisen Schreiben von Zeichendaten in eine Datei
- `dump()`, zum Ausgeben einer textlichen Darstellung mehrerer **R**-Objekte
- `dput()`, zur Ausgabe einer textlichen Darstellung eines **R**-Objektes
- `save()`, zum Speichern einer beliebigen Anzahl von **R**-Objekten im Binärformat (möglicherweise komprimiert) in eine Datei.
- `serialize()`, zum Konvertieren eines **R**-Objektes in ein Binärformat zur Ausgabe.

7.2 Das readr-Package

Ein explizit für das Einlesen erschaffenes **R** Package ist das reader-Package welches, zum Einlesen von großen Datenfiles gedacht ist. Das Package ersetzt die im vorigen Kapitel beschriebenen Funktionen `read.table()` und `read.csv()` mit den Funktionen `read_table()` bzw. `read_csv()`.

Generell gibt es ein paar Gründe, wieso du lieber auf die Funktionen des `readr`-Packages zurückgreifen solltest, als die Basis-**R** Funktionen zu nutzen.

- Durch den effizienteren Code hinter den Funktionen, ist das Einlesen von Daten um ein Vielfaches schneller, was sich vor allem bei größeren Datenfiles bemerkbar macht.
- Die `readr`-Funktionen sind generell auch reproduzierbar. Während die Base-**R** Funktionen Eigenheiten des jeweiligen Betriebssystems des Computers übernehmen, sind `readr`-Funktionen System-agnostisch.

7.3 Speichern von Daten

Wie bereits zu Beginn des Kapitels dargestellt, existieren analog zu den Funktionen, welche Daten in **R** einlesen Funktionen, welche Daten exportieren bzw. abspeichern. Wie jede Funktion in **R**, denen wir bisher begegnet sind, haben auch diese Argumente einen bestimmten Einfluss auf die Art und Weise wie die Funktion durchgeführt wird.

Um nun eine Liste `x` beispielhaft als CSV-Datei abzuspeichern, musst du folgendes machen:

```
write.csv(x, file = "beispiel.csv", row.names = FALSE)
```

Der Aufbau der Argumente hat folgende Struktur. Zunächst solltest du `write.csv()` den Namen des **R**-Objektes geben, das du gespeichert haben möchtest. Als nächstes solltest du einen Dateinamen für deine Datei angeben. **R** wird diesen Namen ziemlich wörtlich nehmen, also stell sicher, dass du eine Erweiterung an den Namen anhängst (.csv). **R** wandelt deine Liste dann in eine Klartextdatei in eine CSV-Datei um, in welcher Werte durch Komma getrennt sind und speichert die Datei in deinem Arbeitsverzeichnis. Um zu sehen, wo sich dein Arbeitsverzeichnis befindet, führe getwd() aus. Um den Speicherort deines Arbeitsverzeichnisses zu ändern, besuche Session > Arbeitsverzeichnis festlegen > Verzeichnis wählen in der Menüleiste von RStudio.

7.4 Daten Selektion

Es ist oft wichtig für bestimmte Operationen nur einen Teil der Daten, also ein Subset, zu betrachten und zu bearbeiten. Diese sogenannte Subsettig von **R** Objekten kann in Base-**R** mithilfe von drei verschiedenen Operatoren gemacht werden. Diese drei Operatoren sind `[]`, `[[]]` und `$`. Der `[]`-Operator selektiert ein Teil der Daten und gibt das Subset in derselben Objekt Klasse aus wie das Originale. Der `[[]]`-Operatoren hingegen kann nur einzelne Elemente aus Listen oder Dataframes subsetten und das resultierende Objekt ist nicht unbedingt aus derselben Objekt Klasse wie das übergeordnete. Hierbei sei daraufhingewiesen, dass **R** bei den Positionen von Elementen in Objekten nicht bei 0 anfängt zu zählen wie andere Programmiersprachen, sondern bei 1. Schlussendlich kann der `$`-Operator genutzt werden um Elemente durch Nutzung ihres Names auszulesen. Zuerst betrachten wir genauer wie man Subsetting eines Vektor macht.

7.4.1 Subsetting von Vektoren

Zu Beginn die Anwendung des `[` Operators auf Vektoren. Hierbei kann man sehen, dass entweder einzelne Elemente ausgelesen werden können, ganze Folgen oder bestimmte beliebige Positionen.

Um einzelne Elemente auszuwählen, kannst du einfach die jeweilige Position im Vektor in den Klammern angeben. Hierfür schauen wir uns einen Beispielvektor an, welcher die Namen beliebige Farben beinhaltet. Um nun ein bestimmtes Element auszuwählen, gebe wir einfach die jeweilige Position innerhalb des Vektors an.

```
foo <- c("rot", "gruen", "blau", "blau", "gelb", "rot")

# Nur das erste Element des Vektors
foo[1]
```

```
## [1] "rot"
```

Du bist ebenfalls in der Lage mehrere Elemente auszuwählen. So kannst du eine Sequenz auswählen `a:b` oder gewisse Werte, wobei du hierfür einen Vektor zu auswahl übergeben musst, sodass du innerhalb des `[]`-Operators ein Vektor der Positionen hast `c(1, ..., n)`.

```
# Die ersten vier Elemente
foo[1:4]
```

```
## [1] "rot"   "gruen" "blau"  "blau"
```

Es gibt auch einige hilfreiche Kniffe, welche es dir erlauben bestimmte Positionen aus dem Vektor auszulesen. Um z.B. den letzten Eintrag auszuwählen übergeben wir einen Auswahlvektor, welcher `length(name des vektors)` nutzt. Der Befehl `length()` gibt an was die Länge eines Vektors ist und somit wie viele Einträge dieser besitzt

```
# Der erste, dritte und letzte Eintrag
foo[c(1, 3, length(foo))]
```

```
## [1] "rot"  "blau" "rot"
```

7.4.2 Subsetting von Listen

Listen können in **R** in drei verschiedenen Arten gesubsettet werden. Hierfür nutzt man die oben genannten drei verschiedenen Operatoren. Um die drei verschiedenen Arten zu illustrieren erschaffen wir zuerst eine Beispielsliste

```
x <- list(foo = 1:3, faa = "hallo", faz = 0.3)
x
```

```
## $foo
## [1] 1 2 3
##
## $faa
## [1] "hallo"
##
## $faz
## [1] 0.3
```

Eine der einfachsten und bequemsten Arten Listen zu subsetten ist die Benutzung des `$`-Operators. Der Grund hierfür ist, dass wenn du den Namen der Zielliste eingibst, gefolgt vom `$`-Operator, kannst du durch das Betätigen der Tabulator Taste Rstudios Autovervollständigung nutzen. Dabei werden alle Elemente einer Liste aufgeführt und du kannst einfach das gewünschte Element über die Pfeiltasten auswählen.

```
x$foo
```

```
## [1] 1 2 3
```

```
x$faa
```

```
## [1] "hallo"
```

Ähnlich zum Subsetten der Matrizen und Vektoren, können Listen ebenso mit dem `[]`- bzw. `[[]]`-Operator gesubsettet werden. Wenn du also das erste Element der Liste auswählen willst kannst du einfach wie in folgendem Beispiel machen

```
x[[1]]
```

```
## [1] 1 2 3
```

Ebenso ist es wieder möglich in Kombination mit dem `[[]]`-Operator mit Hilfe der Namen der Elemente die Liste zu subsetten.

```
x[["foo"]]
```

```
## [1] 1 2 3
```

Schlussendlich ist es ebenso möglich mehrere Elemente der Liste auszuwählen. Hierfür nutze einfach den `[]` oder `[[]]`-Operator und setze gebe einen Vektor an, welche die Elemente bestimmt. Allerdings sei hierbei darauf hingewiesen, dass je nach dem welchen der beiden Operatoren du nutzt verschieden Elemente ausgewählten werden. Zum einfacheren Verständnis das folgende Beispiel

```
# Das Zweite und das dritte Element der Liste
x[c(2,3)]
```

```
## $faa
## [1] "hallo"
##
## $faz
## [1] 0.3
```

```
# Der dritte Eintrag des ersten Elements
x[[c(1,3)]]
```

```
## [1] 3
```

7.4.3 Subsetting von Matrizen

Das Subsetten einer Matrix erfolgt nach denselben Regel wie das Subsetten der Vektoren über den `[]`-Operator. Statt nur einen Indizierungsvektor zu verwenden, verwenden wir jetzt zwei Indizierungsvektoren: einen für die Zeilen und einen für die Spalten. Dazu verwendest du die Notation `matrix[rows,`

columns], wobei Zeilen und Spalten Vektoren von ganzen Zahlen sind. Hier ein Beispiel anhand der im Kapitel 3 erschaffenen Matrix was damit genau gemeint ist

```
m <- matrix(c(1,2,3,4,5,6), ncol = 2, nrow = 3, byrow = TRUE)
m
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
## [3,]    5    6
```

```
# Abfrage des Elements in der zweiten Reihe und der ersten Spalte
m[2,1]
```

```
## [1] 3
```

Wenn man nun allerdings anstatt eines Elementes, alle Elemente einer Reihe oder Spalte abfragen will kann man die jeweilige Row bzw. Column Angabe frei lassen. Beispiel hier

```
# Alle Elemente der ersten Reihe
m[1, ]
```

```
## [1] 1 2
```

```
# Alle Elemente der ersten Spalte
m[ , 1]
```

```
## [1] 1 3 5
```

```
# Die ersten zwei Reihen
m[c(1,2), ]
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

7.4.4 Subsetting von DataFrames

Genau wie bei Vektoren kannst du auf bestimmte Daten in DataFrames mit Hilfe von Klammern `[ ]` zugreifen. Statt nur einen Indizierungsvektor zu verwenden, verwenden wir jetzt zwei Indizierungsvektoren: einen für die Zeilen und einen für die Spalten. Dazu verwendest du die Notation `data[rows, columns]`, wobei Zeilen und Spalten Vektoren von ganzen Zahlen sind.

Als Beispiel nehmen wir einen Dataframe namens `df` an. Wir wollen nun jeweils einmal die erste Reihe, dann die zweite Spalte und dann die Elemente haben welche in den ersten drei Reihen und der zweiten Spalte sind, auswählen. Hierfür musst du einfach wie folgt vorgehen

```
# Erste Reihe
df[1, ]

# zweite Spalte
df[, 2]

# Ersten drei Reihen und zweite Spalte
df[1:3, 2]
```

Um dir diese Herangehensweise anhand eines Beispiels zu erklären, nutzen wir ab sofort für den Rest dieses Kapitels den R-Datensatz „airquality“, welcher Messung der Luftqualität in New York darstellt.

```
head(airq)
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
## 2    36     118  8.0   72     5   2
## 3    12     149 12.6   74     5   3
## 4    18     313 11.5   62     5   4
## 5    NA      NA 14.3   56     5   5
## 6    28      NA 14.9   66     5   6
```

Um also nun alle Elemente der ersten Zeile auszuwählen nutzt man einfach den Namen des DateFrames und setzt in die `[ ]`-Operatoren an die erste Stelle eine `1` und lässt die zweite Stelle frei.

```
airq[1, ]
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
```

Ebenso wie bei anderen Datensturkturen ist es möglich mehrere Zeilen oder Spalten Werte durch einen Vektor auszuwählen, so um z.B die erste, zweite und vierte Zeile auszuwählen übergibt man den Vektor `c(1,2,4)`.

```
airq[c(1,2,4), ]
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
## 2    36     118  8.0   72     5   2
## 4    18     313 11.5   62     5   4
```

Es ist ebenso möglich einzelne Variablen eines Dataframes abzufragen in dem du den `$`-Operator nutzt. So kannst du die Variable Temperatur (`Temp`) des Datensatzes der Luftqualität abrufen, indem du den Namen des Dataframes gefolgt vom `$`-Operator und dem Namen der Variable eingibst.

```
airq$Temp
```

```
##   [1] 67 72 74 62 56 66 65 59 61 69 74 69 66 68 58 64 66 57 68 62 59 73 61 61 57
##  [26] 58 57 67 81 79 76 78 74 67 84 85 79 82 87 90 87 93 92 82 80 79 77 72 65 73
##  [51] 76 77 76 76 76 75 78 73 80 77 83 84 85 81 84 83 83 88 92 92 89 82 73 81 91
##  [76] 80 81 82 84 87 85 74 81 82 86 85 82 86 88 86 83 81 81 81 82 86 85 87 89 90
## [101] 90 92 86 86 82 80 79 77 79 76 78 78 77 72 75 79 81 86 88 97 94 96 94 91 92
## [126] 93 93 87 84 80 78 75 73 81 76 77 71 71 78 67 76 68 82 64 71 81 69 63 70 77
## [151] 75 76 68
```

Die Indizierung von DataFrames mit logischen Vektoren ist fast identisch mit der Indizierung numerischer Vektoren. Zuerst erstellen wir einen logischen Vektor, der nur WAHR- und FALSCH-Werte enthält. Als Nächstes indizieren wir einen DataFrame (typischerweise die Zeilen) unter Verwendung des logischen Vektors, um nur Werte zurückzugeben, für die der logische Vektor WAHR ist. Um zum Beispiel ein neues Objekt zu erstellen welches nur die Elemente des Dataframes übernimmt bei denen der Wert der Variable Temperatur (`Temp`) über einen gewissen Wert steigt z.B. `> 95`. So kann man dies wie folgt machen

Zu erst erstellt man eben einen logischen Vektor der aussagt ob `airq$Temp > 95`

```
airq$Temp > 95
```

```
##   [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
##  [49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##  [97] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [109] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE
## [121] FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [145] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

Wir fügen diesen Vektor in die Abfrage über die `[ ]`-Operatoren.

```
airq[airq$Temp > 95, ]
```

```
##     Ozone Solar.R Wind Temp Month Day
## 120    76     203  9.7   97     8  28
## 122    84     237  6.3   96     8  30
```

–>

Chapter 8

Plotten

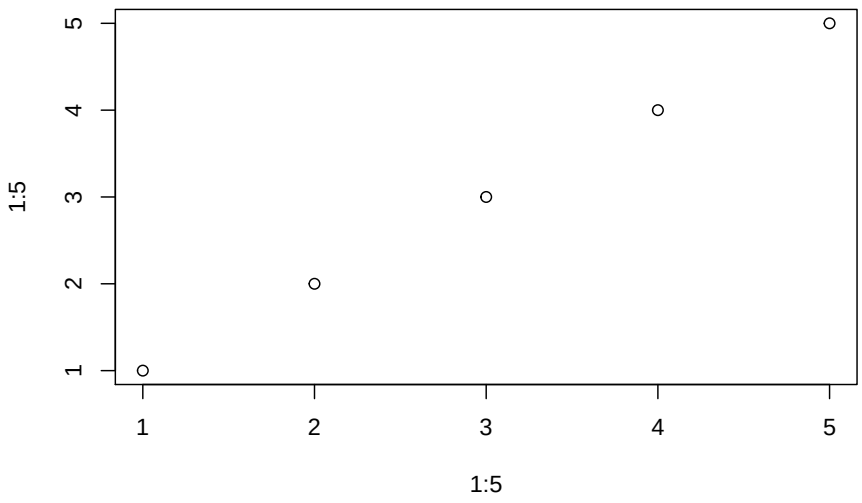
Oft ist es wichtig Zusammenhänge nicht nur in Zahlen, sondern ebenso graphisch darzustellen. Das folgende Kapitel zeigt dir wie man verschiedene Arten von wichtigen grundlegenden Plots in **R** erstellt und wie man diese in einer Art und Weise kreiert, sodass man sie ohne Probleme in Seminar- oder Abschlussarbeiten nutzen kann.

Dieses Kapitel wird sich nur mit den in der base-**R** Version enthaltenen Plot-Funktionen befassen. Wir lassen bewusst **R**-Packages wie `ggplot2` oder `plotly` aus, solltest du dich allerdings für die Funktionalität dieser Packages und der damit verbundenen Verbesserung der Möglichkeiten der graphischen Darstellung interessieren, empfehlen wir dir die https://socviz.co/ und https://rkabacoff.github.io/datavis/ .

8.1 Plot-Funktion

Die einfachste Möglichkeit einen Plot in **R** zu erstellen, ist die **R**-base Funktion `plot()` zu nutzen. In der Standard Einstellung erstellt der Befehl ein Streudiagramm, basierend auf den beiden Input Vektoren für die Abszissen- sowie der Ordinatenwerte.

```
plot(x = 1:5,
     y = 1:5)
```

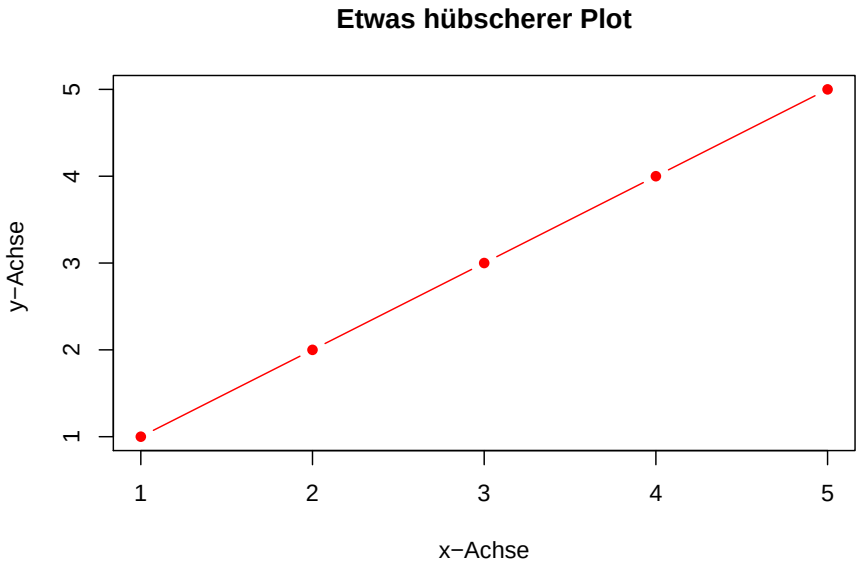


Erklärung der wichtigsten Argumente der `plot()`-Funktion

Argument	Beschreibung
`x, y`	Vektoren gleicher Länge, die die x- und y-Werte der Punkte angeben
`type`	Art der Handlung. `"l"` bedeutet Linien, `"p"` bedeutet Punkte, `"b"` bedeutet Linien und Punkte, `"n"` bedeutet kein Plotten
`main`, `xlab`, `ylab`	Strings, die den Titel der Handlung beschriften, und x- und y-Achsen
`xlim`, `ylim`	Grenzen zu den Achsen. Zum Beispiel setzt `xlim = c(0, 100)` das Minimum und das Maximum der x-Achse auf 0 und 100.
`pch`	Eine ganze Zahl, die den Typ der Zeichensymbole angibt oder eine Zeichenkette.
`col`	Hauptfarbe der Plottsymbole. Zum Beispiel wird `col = "rot"` rote Symbole erzeugen.
`cex`	Ein numerischer Vektor, der die Größe der Symbole angibt (von 0 bis Inf). Die Standardgröße ist 1. `cex = 4` macht die Punkte sehr groß, während `cex = .5` sie sehr klein macht.

```
plot(x = 1:5,
     y = 1:5,
     type = "b",
     main = "Etwas hübscherer Plot",
```

```
     xlab = "x-Achse",
     ylab = "y-Achse ",
     col = "red",
     pch = 16,
     cex = 1)
```



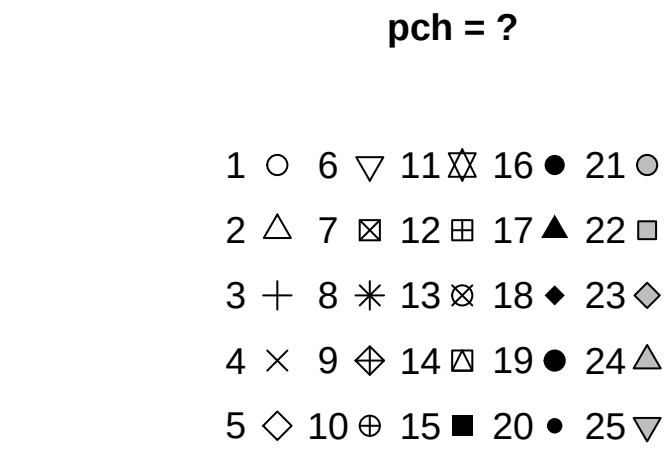
Generell gilt es hier, wie bei allen anderen Funktionen in **R**, wenn du mehr Information über die Art und Weise der Anwendung von **R**-Funktionen möchtest einfach die `?`-Funktion in die Console eingeben oder über den View-Tab in RStudio zu der Funktionsbeschreibung zu gelangen.

8.2 Symbole

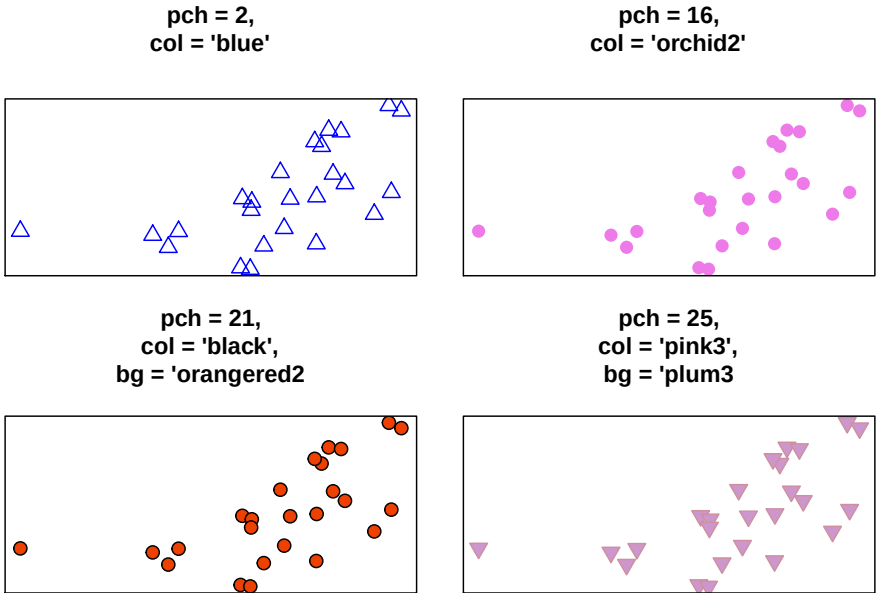
Eine der wichtigsten Optionen bei der Anpassung von Grafiken in **R** ist die Wahl der Symbole um Datenpunkte darzustellen. Das dafür genutzte Argumente `pch` erlaubt eine breite Auswahl an verschiedensten Formen an Symbolen. Um ein Symboltyp über `pch` auszuwählen gibt es zwei verschiedene Arten. Entweder durch die Angabe einer Zahl oder einer Zeichenfolge.

Symbole unterscheiden sich in ihrer Form und in der Art wie sie eingefärbt sind. Die Symbole 1 bis 14 haben nur einen Rand und sind immer leer, während die Symbole 15 bis 20 keinen Rand haben und immer gefüllt sind. Die Symbole 21 bis 25 haben sowohl einen Rand als auch eine Füllung. Um die Rahmenfarbe

oder den Hintergrund für die Symbole 1 bis 20 festzulegen, verwende das Argument `col`. Bei den Symbolen 21 bis 25 legst du die Farbe des Rahmens mit `col` und die Farbe des Hintergrunds mit `bg` fest.



Hier ein Beispiel dafür, wie verschiedene Symbole und Farben Grafiken verändern können:



8.3 Low-Level Plot-Funktionen

Die sogenannten Low-Level-Plot Funktionen erlauben es ein Element eines Plots zu verändern oder neue hinzuzufügen. Die wichtigsten und am meisten genutzten sind hierbei die folgenden

Funktion	Ergebnis
`points(x, y)`	Fügt Punkte hinzu
`abline()`, `segments()`	Fügt Linien oder Segmente hinzu
`arrows()`	Fügt Pfeile hinzu
`curve()`	Fügt eine Kurve hinzu, die eine Funktion darstellt
`rect()`,`polygon()`	Fügt ein Rechteck oder eine willkürliche Form hinzu
`text()`, `mtext()`	Fügt Text innerhalb der Handlung oder an den Rändern der Handlung hinzu
`legend()`	Fügt eine Legende hinzu
`axis()`	Fügt eine Achse hinzu

Wenn man die Low-Level-Plot Funktionen verstehen will, macht es Sinn mit einem leeren Plot anzufangen.

```r
plot(x = 0,
     xlab = "X-Achse",
     ylab = "Y-Achse",
     xlim = c(0, 100),
     ylim = c(0, 100),
     main = "Leerer Plot",
     type = "n")
```



8.4 points() und Segments()

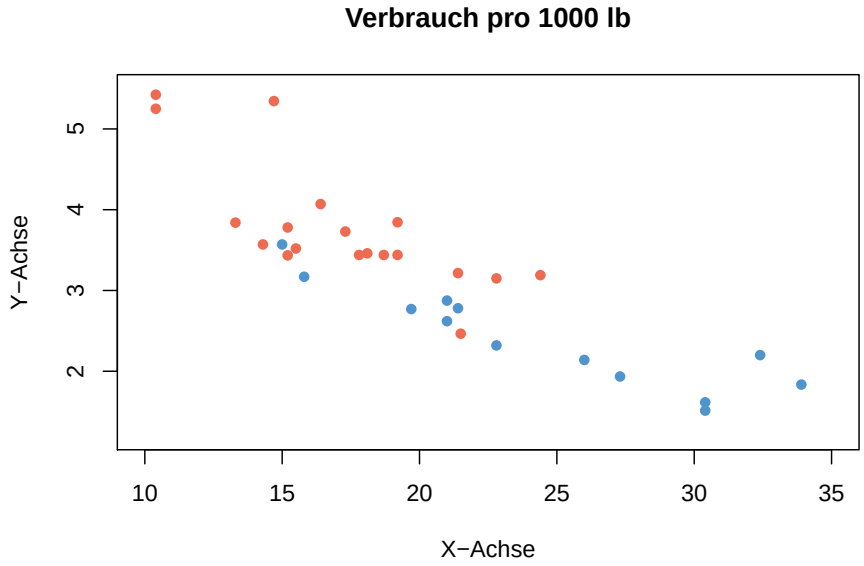
Um neue Punkte zu einem bestehenden Diagramm hinzuzufügen, verwende die Funktion `points()`. Die Funktion `points()` hat viele ähnliche Argumente wie die Funktion `plot()`, wie `x` (für die x-Koordinaten), `y` (für die y-Koordinaten) und Parameter wie `col` (Randfarbe), `cex` (Punktgröße) und `pch` (Symboltyp).

Ein Beispiel anhand des vorher schon genutzen `mtcars`-Datenset. Für nähere Informationen einfach `?mtcars` eingeben. Hierbei wollen wir alle Autos hinsichtlich ihres Verbrauches in Miles-per-Gallon in Relation zu ihrem Gewicht abbilden. Dabei wollen wir die Autos je nach Gangschaltungstyp (Automatik und Manuell) farblich kennzeichen.

```
plot(x = 0,
     xlab = "X-Achse",
     ylab = "Y-Achse",
     xlim = c(10, 35),
     ylim = c(1.2, 5.5),
     main = "Verbrauch pro 1000 lb",
     type = "n")

# Punkte hinzufügen Automatik
points(x = mtcars$mpg[mtcars$am == 0],
```

```
       y = mtcars$wt[mtcars$am == 0],
       pch = 16,
       col = "coral2")
# Punkte hinzufügen Manuell
points(x = mtcars$mpg[mtcars$am == 1],
       y = mtcars$wt[mtcars$am == 1],
       pch = 16,
       col = "steelblue3")
```



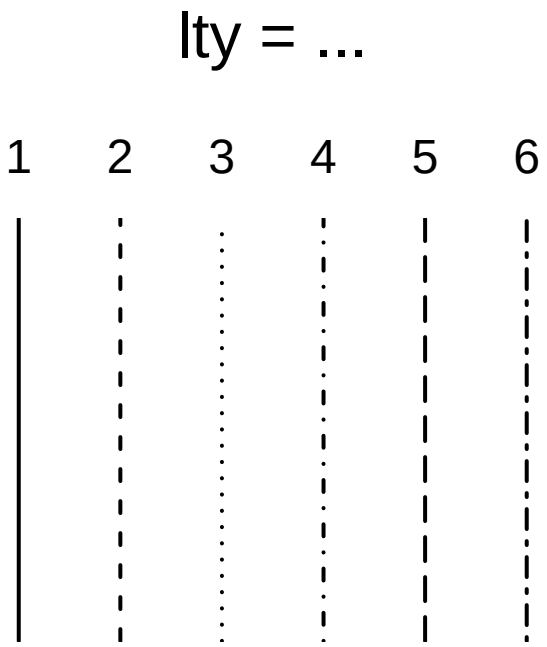
8.5 abline() und segments()

Um einem Plot gerade Linien hinzuzufügen, verwendest du `abline()` oder `segments()`. `abline()` fügt eine Linie über den gesamten Plot hinzu, während `segments()` eine Linie mit definierten Anfangs- und Endpunkten hinzufügt.

Argument	Ergebnis
`h, v`	Positionen der horizontalen und vertikalen Linien (nur für `abline()`)
`x0, y0, x1, y1`	Anfangs- und Endkoordinaten der Linien (nur für `segments()`)
`lty`	Linientyp. 1 = durchgezogen, 2 = gestrichelt, 3 = gepunktet, ...

Argument	Ergebnis
`lwd`	Breite der Linien, die durch eine Zahl angegeben wird
`col`	Line Farbe

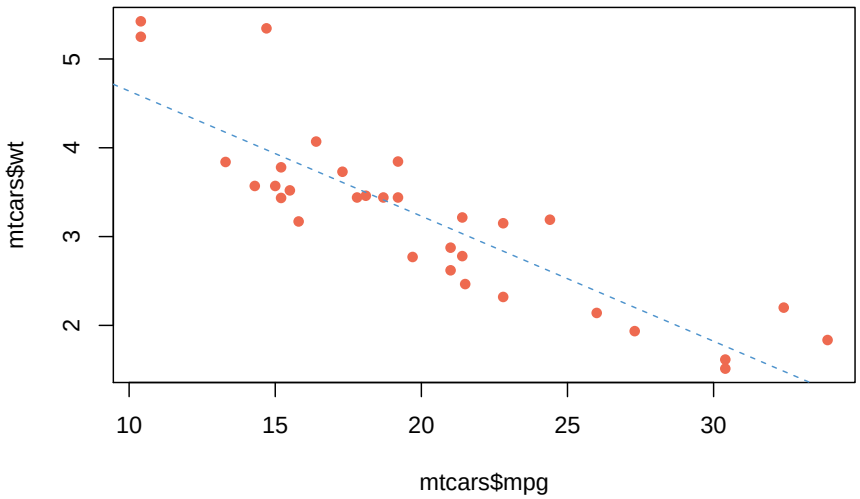
Um das Aussehen der Linien zu ändern, nutzt man das Argument `lty`.



Du kasnnst auch eine Regressionslinie (auch als Linie der besten Anpassung bezeichnet) zu einem Scatterplot hinzufügen, indem du ein Regressionsobjekt eingibst, das mit `lm()` als Hauptargument für `abline()` erstellt wurde.

```
plot(x = mtcars$mpg,
     y = mtcars$wt,
     pch = 16,
     col = "coral2")

# Regressionslinie hinzufügen
abline(lm(wt ~ mpg, data = mtcars),
       lty = 2,
       col = "steelblue3")
```



8.6 legend()

Die letzte Low-Level-Plotting-Funktion, die wir im Detail besprechen werden, ist `legend()`, die eine Legende zu einem Plot hinzufügt.

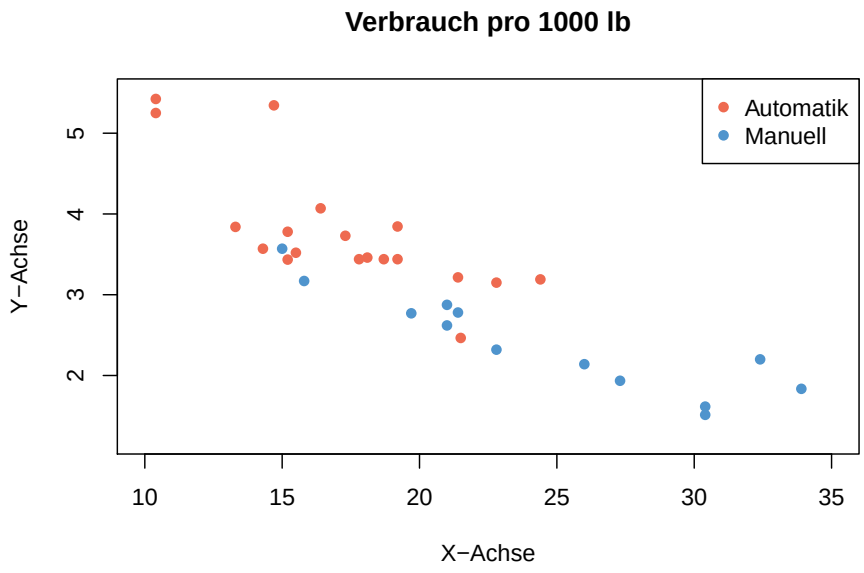
Argument	Ergebnis
`x, y`	Die Koordinaten der Legende - z.B. `x = 0, y = 0` setzen den Text an die Koordinaten (0, 0).
`labels`	Ein String-Vektor, der den Text in der Legende angibt. Zum Beispiel wird 'legend = c("Männlich,"Weiblich")

Es ist ebenfalls möglich die Position der Legende durch Worte einzugeben wie `"topright"`, `"topleft"`. Zum Beispiel wird "unten rechts" die Legende immer in der rechten unteren Ecke der Handlung platziert. Ein Beispiel an unserem Auto Datenset:

```
plot(x = 0,
     xlab = "X-Achse",
     ylab = "Y-Achse",
     xlim = c(10, 35),
```

```r
     ylim = c(1.2, 5.5),
     main = "Verbrauch pro 1000 lb",
     type = "n")

# Punkte hinzufügen Automatik
points(x = mtcars$mpg[mtcars$am == 0],
       y = mtcars$wt[mtcars$am == 0],
       pch = 16,
       col = "coral2")
# Punkte hinzufügen Manuell
points(x = mtcars$mpg[mtcars$am == 1],
       y = mtcars$wt[mtcars$am == 1],
       pch = 16,
       col = "steelblue3")
# Legende hinzufügen
legend("topright",
       legend = c("Automatik", "Manuell"),
       col = c('coral2', 'steelblue3'),
       pch = c(16, 16),
       bg = "white")
```



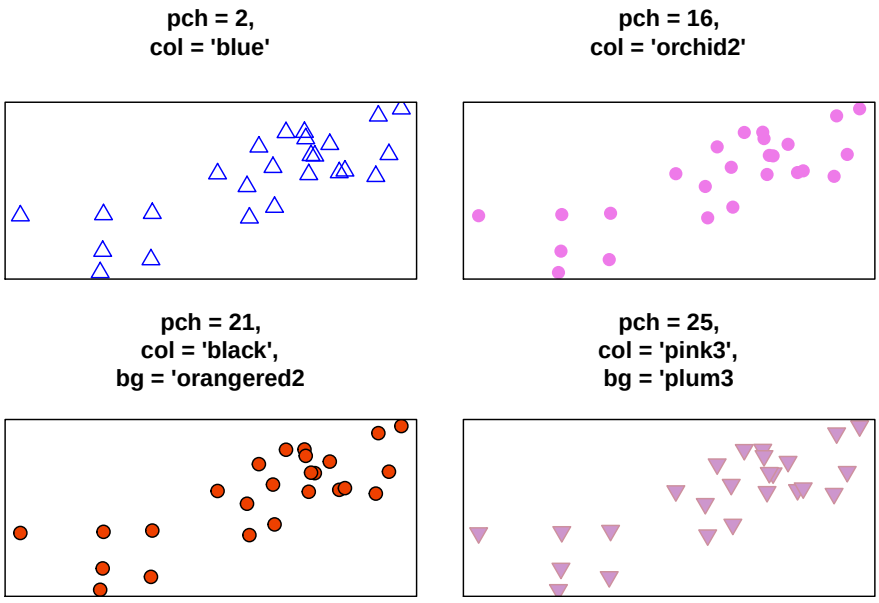
8.7 Multiple Plots

Wie du im vorigen Beispiel gehen hast, ist es ebenfalls möglich mehrere Plots in einer einzelnen Grafik abzubilden. Hier für gibt's es bei Base-R Plots zwei Wege dieses zu erreichen. Am häufigsten wird dieses mit dem Parameter `par(mfrow)` und `par(mfcol)` erreicht. Gehen wir diese die beiden Funktionen durch. Mit den Parametern mfrow und mfcol kannst du eine Matrix von Plots in einem Plotraum erstellen. Beide Parameter nehmen einen Vektor der Länge zwei als Argument, also die Anzahl der Zeilen und Spalten in der resultierenden Plottmatrix.

Was ist also der Unterschied zwischen `par(mfrow)` und `par(mfcol)`? Der einzige Unterschied besteht darin, dass `par(mfrow)` sequentielle Plots zeilenweise in die Plottmatrix einfügt, während `par(mfcol)` sie spaltenweise füllt.

Der folgende Code erstellt beispielsweise die 2 x 2-Plotting-Matrix des vorigen Beispiels.

```
# Es soll eine 2 x 2 Plotmatrix enstehen:
par(mfrow = c(2, 2))
par(mar = c(0, 1, 6, 1))
# Daten erschaffen
x.data <- rnorm(25)
y.data <- x.data + rnorm(25)
# Plot 1
plot(x = x.data, y = y.data, xaxt = "n", yaxt = "n", xlab = "", ylab = "", main = "pch = 2,\
     pch = 2, col = "blue", cex = 1.5, cex.main = 1.2)
# Plot 2
plot(x = x.data, y = y.data, xaxt = "n", yaxt = "n", xlab = "", ylab = "", main = "pch = 16,
     pch = 16, col = "orchid2", cex= 1.5, cex.main = 1.2)
# Plot 3
plot(x = x.data, y = y.data, xaxt = "n", yaxt = "n", xlab = "", ylab = "", main = "pch = 21,
     cex= 1.5, cex.main = 1.2,
     pch = 21, col = "black", bg = "orangered2")
# Plot 4
plot(x = x.data, y = y.data, xaxt = "n", yaxt = "n", xlab = "", ylab = "", main = "pch = 25,
     cex= 1.5, cex.main = 1.2,
     pch = 25, col = "pink3", bg = "plum3")
```


pch = 2,
col = 'blue'
pch = 16,
col = 'orchid2'
pch = 21,
col = 'black',
bg = 'orangered2
pch = 25,
col = 'pink3',
bg = 'plum3